A1.1 REQUERIMIENTOS PARA WINDOWS

PROGRAMA EDUCATIVO

Ingeniería en Sistemas Computacionales

NOMBRE DEL PROYECTO

Automatización de mantenimiento preventivo con monitoreo permanente utilizando APIs y scripts

EMPRESA

Infraestructura Digital Predictiva Bitlab, S.A.P.I. de C.V.

PRESENTA

Axel Gabriel Ramírez Ortiz
Matrícula: 122043202 · Grupo: S206

ASESOR UPQ:

Ana Laura Lira Cortes

CO-ASESOR UPQ:

Ivan Peredo Valderrama

ASESORA EMPRESARIAL:

M.T.A. Sandra Lucía de la Fuente Bermúdez

Querétaro, Qro., 29 de octubre del 2026

ÍNDICE

1. OBJETIVO DE LA ACTIVIDAD

2. DESARROLLO DE LA ACTIVIDAD

- Versiones de Windows
- PowerShell
- Política de ejecución
- Permisos y cuenta de ejecución
- Programador de tareas
- Flujo del agente en Windows
- Archivos, red y antivirus

3. RESULTADOS DE LA ACTIVIDAD

4. PENDIENTES Y SIGUIENTES PASOS

REFERENCIAS

1. OBJETIVO DE LA ACTIVIDAD

Con esta actividad busqué dejar claro todo lo que necesita el agente para trabajar en las computadoras con Windows de los clientes de Bitlab-IT. Antes de escribir una sola línea de los scripts, tenía que saber en qué versiones de Windows iba a funcionar, con qué herramientas lo iba a construir, con qué permisos se iba a ejecutar y cómo iba a correr solo, sin que un técnico tuviera que conectarse a cada equipo.

Esta actividad responde a la parte de Windows de la primera pregunta del proyecto: ¿qué revisiones y tareas de mantenimiento se pueden hacer de forma automática en Windows y en Mac, con resultados comparables entre ambos? Lo que aquí se define es la base de los scripts (entregable E2) y del instalador con su guía (entregable E3).

Tabla 1. Datos de la actividad.

Concepto	Descripción
Clave de la actividad	A1.1
Fase del protocolo	F1. Requerimientos y arquitectura
Semanas	S1 – S2 (7 al 18 de septiembre de 2026)
Horas planeadas	24 horas
Producto	Requerimientos Windows
Entregable al que pertenece	E1. Documento de requerimientos y arquitectura (sub-entregable E1.1)

Fuente: Protocolo de estadía profesional, Bitlab-IT (2026).

2. DESARROLLO DE LA ACTIVIDAD

Para levantar estos requerimientos revisé el protocolo de estadía, la documentación de Microsoft sobre PowerShell y el Programador de tareas, y el agente de prueba que escribí para la demostración de la consola remota. En cada punto comparé las opciones que tenía y elegí la que mejor se ajustaba a lo que pide el proyecto: usar solo lo que ya trae el sistema, mantener la seguridad y no complicarle la instalación al técnico.

Versiones de Windows

Lo primero fue decidir en qué versiones debía funcionar el agente. En clínicas y hospitales es muy común encontrar computadoras de distintas épocas conviviendo en la misma red, así que no podía pensar solo en la versión más nueva. Las agrupé en tres niveles, como se ve en la Tabla 2.

Tabla 2. Versiones de Windows y su nivel de soporte en el proyecto.

Versión	Nivel de soporte	Por qué
Windows 11 (23H2, 24H2 o posterior)	Soportada. Es la versión principal.	Trae PowerShell y el Programador de tareas, y es la versión con la que se harán las pruebas de laboratorio.
Windows 10 22H2	Soportada con reserva	Microsoft dejó de darle soporte general el 14 de octubre de 2025, pero sigue siendo muy común en clínicas. Se incluye, recomendando al cliente actualizar o contratar las actualizaciones extendidas.
Windows Server 2016, 2019 y 2022	Soportada, por validar en el piloto	Son los servidores que suelen tener las clínicas para imágenes o laboratorio.
Windows 7, 8 y 8.1	No soportada	Ya no reciben actualizaciones de seguridad y no garantizan una conexión segura con la plataforma.

Fuente: Elaboración propia.

El inventario real de lo que tienen los clientes lo podré completar cuando se tenga acceso a la plataforma de monitoreo de Bitlab-IT.

PowerShell

La regla del protocolo es no depender de programas externos, y eso me llevó directamente a **Windows PowerShell 5.1**. Viene instalado en Windows 10, Windows 11 y Windows Server 2016 en adelante, así que el agente puede funcionar sin instalar nada extra. Consideré PowerShell 7, que es más moderno, pero habría que instalarlo en cada equipo, y eso va en contra del protocolo y le suma un paso al técnico (Microsoft, 2024a).

Al revisarlo con más detalle encontré cuatro cuidados que el agente tiene que tener. El primero es usar solo funciones que existen en la versión 5.1, como `Invoke-RestMethod`, `ConvertTo-Json`, `Get-CimInstance` y `Get-Process`; el agente de la demostración ya sigue esa regla. El segundo es activar TLS 1.2 al iniciar, porque

Supabase no acepta conexiones más antiguas. El tercero es mandar todo en UTF-8 para no perder acentos ni la ñ.

El cuarto me pareció el más importante: Windows en español traduce el texto de comandos como ping o ipconfig. Si el agente sacara las métricas leyendo ese texto, dejaría de funcionar en cuanto un equipo estuviera en otro idioma. Por eso las métricas se toman directamente del sistema, y el texto solo se usa para mostrarle al técnico lo que respondió el equipo.

Política de ejecución

Windows tiene una regla llamada **política de ejecución** que decide si se pueden correr scripts. En las computadoras de escritorio viene como **Restricted**, que no deja ejecutar ninguno, y en los servidores como **RemoteSigned**. Tenía que encontrar la forma de que el agente pudiera trabajar sin cambiar la configuración de todo el equipo. Estas fueron las opciones que analicé:

Tabla 3. Opciones evaluadas para la política de ejecución.

Opción	Cómo funciona	Decisión
Permitir scripts solo al agente	La tarea abre PowerShell con -ExecutionPolicy Bypass, que aplica únicamente a ese proceso.	La elegí para el laboratorio y el piloto. Es como ya funciona la demostración.
Firmar los scripts	Los scripts llevan la firma de un certificado de Bitlab-IT y el equipo confía en ella.	La recomiendo para producción, pero requiere un certificado y la aprobación de la asesora externa.
Cambiar la política de todo el equipo	Se modifica la configuración general de Windows.	La descarté porque afecta a todo el equipo y no solo al agente.

Fuente: Elaboración propia.

Hay un caso que conviene tener presente: si el área de sistemas del cliente fija la política con directivas de grupo, esa configuración gana sobre la opción del agente. En esos equipos habrá que coordinarse con el cliente o usar scripts firmados.

Permisos y cuenta de ejecución

No todo lo que hace el agente necesita los mismos permisos. Leer cuánta memoria se está usando lo puede hacer cualquier usuario, pero reiniciar la cola de impresión o limpiar los temporales del sistema ya requiere permisos de administrador. La Tabla 4 muestra lo que necesita cada tipo de operación.

Tabla 4. Permiso mínimo requerido por tipo de operación.

Operación	Ejemplo	Permiso
Leer el uso de procesador, memoria y disco	`Get-CimInstance, Get-Process`	Usuario estándar
Leer los registros del sistema	`Get-WinEvent -LogName System`	Usuario estándar
Ver con detalle los procesos de otros usuarios	Ruta y dueño de un proceso	Administrador
Reiniciar un servicio	`Restart-Service Spooler`	Administrador
Limpiar los temporales del sistema	Carpeta `C:\Windows\Temp`	Administrador
Reiniciar el equipo	`shutdown /r /t 300`	Administrador. Modifica el equipo.

Fuente: Elaboración propia.

Con esto en la mesa, considero que la mejor opción es que el agente se ejecute con la cuenta **SYSTEM** de Windows. Tiene los permisos que piden las acciones del catálogo y funciona aunque nadie haya iniciado sesión, que es justo lo que pasa en muchas computadoras de recepción o de enfermería. También pensé en una cuenta local con permisos mínimos, pero no podría reiniciar servicios ni limpiar temporales, así que el agente se quedaría a medias.

Claro que una cuenta con tantos privilegios hay que cuidarla. Por eso el diseño pone tres candados. El agente solo ejecuta comandos que ya están registrados y revisados en el catálogo, nunca texto escrito libremente. Cada computadora tiene su propia clave, guardada en una carpeta que solo pueden leer SYSTEM y los administradores. Y las acciones que modifican el equipo piden confirmación del técnico y tienen un modo de simulación que muestra lo que harían sin aplicar ningún cambio. Esta propuesta la debe validar la asesora externa antes de instalar el agente con clientes.

Programador de tareas

El Programador de tareas es la herramienta de Windows que ejecuta programas en un horario, sin que nadie intervenga (Microsoft, 2024b). Decidí separar el trabajo del agente en dos tareas, porque hacen cosas muy distintas: una revisa el equipo cada cierto tiempo y la otra tiene que estar siempre atenta por si llega un comando.

Tabla 5. Tareas programadas del agente en Windows.

Tarea	Cuándo se ejecuta	Qué hace	Límite
BitlabReporte	Cada 5 minutos, de forma indefinida	Revisa el estado del equipo y envía el reporte a la plataforma.	Menos de 2 minutos por ejecución
BitlabComandos	Al encender el equipo; se reinicia sola si falla	Pregunta cada pocos segundos si hay comandos pendientes, los ejecuta y envía la salida en vivo.	Una sola copia a la vez

Fuente: Elaboración propia.

Así es como el instalador registrará la tarea de comandos:

```
$accion = New-ScheduledTaskAction -Execute 'powershell.exe' `
  -Argument '-NoProfile -NonInteractive -ExecutionPolicy Bypass -WindowStyle Hidden -File "C:\Program
Files\Bitlab\agente.ps1"'
$disparador = New-ScheduledTaskTrigger -AtStartup
$principal  = New-ScheduledTaskPrincipal -UserId 'SYSTEM' -LogonType ServiceAccount -RunLevel Highest
$ajustes    = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries `
  -StartWhenAvailable -RestartCount 3 -RestartInterval (New-TimeSpan -Minutes 1) `
  -ExecutionTimeLimit ([TimeSpan]::Zero) -MultipleInstances IgnoreNew
Register-ScheduledTask -TaskName 'BitlabComandos' -TaskPath '\Bitlab\' -Action $accion `
  -Trigger $disparador -Principal $principal -Settings $ajustes
```

De todas las opciones del código, tres son las que más importan. `IgnoreNew` evita que corran dos copias del agente al mismo tiempo, `RestartCount` hace que la tarea se levante sola si se cae por un error, y las opciones de batería permiten que el agente siga trabajando en laptops desconectadas.

Flujo del agente en Windows

Para tener claro cómo se comporta el agente de comandos desde que se enciende la computadora, armé el diagrama de flujo de la Figura 1. Ahí se ve el ciclo completo: lee su configuración, pregunta si hay trabajo, ejecuta o simula el comando, envía la salida

y vuelve a empezar. Si en algún momento no hay conexión, espera cada vez un poco más antes de volver a intentar.

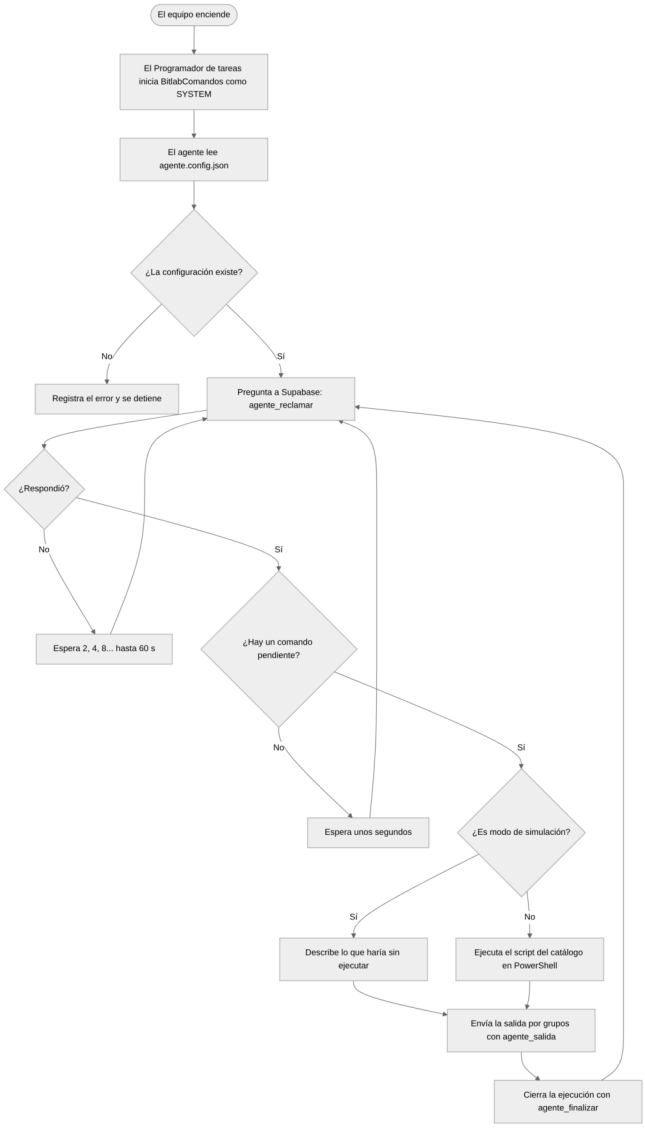


Figura 1. Diagrama de flujo del agente de comandos en Windows. *Fuente: Elaboración propia.*

Este flujo ya funciona en la demostración, con el agente corriendo en mi computadora con Windows.

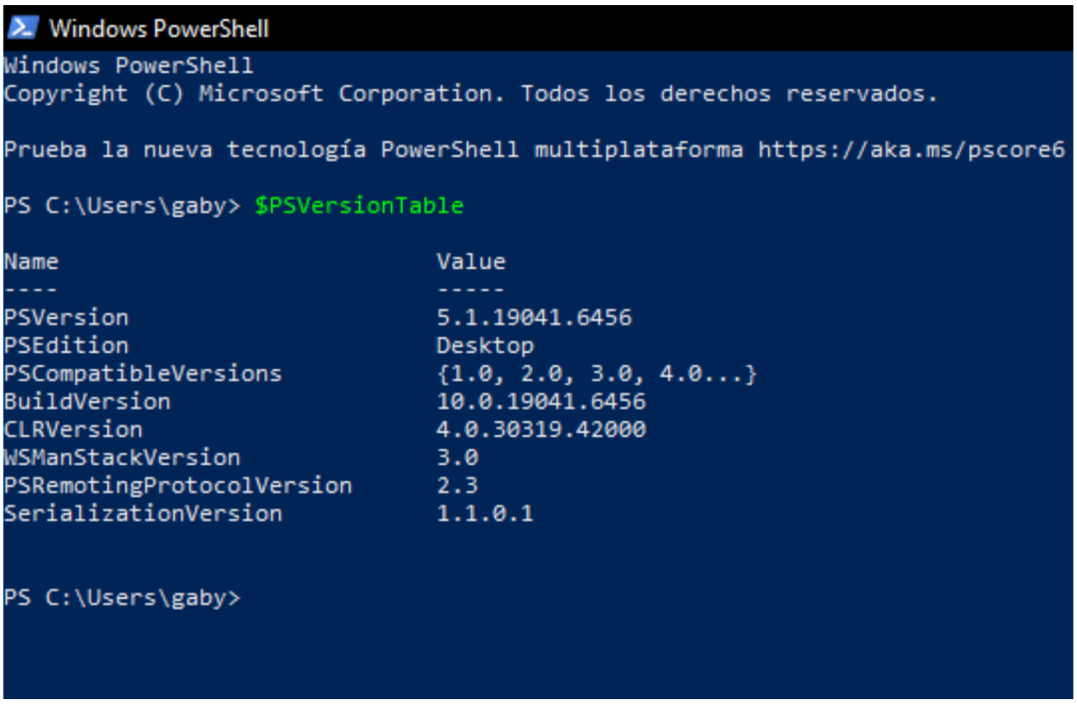


Figura 2. Versión de PowerShell disponible en el equipo Windows de pruebas. *Fuente: Elaboración propia.*

Archivos, red y antivirus

Los scripts irán en C:\Program Files\Bitlab, donde solo los administradores pueden hacer cambios. La configuración con la clave del equipo, los reportes pendientes y el registro de ejecuciones irán en C:\ProgramData\Bitlab, con permisos solo para SYSTEM y administradores. En la demostración la configuración vive junto al script y no se sube a GitHub; el instalador se encargará de moverla a su lugar.

En cuanto a la red, el agente solo necesita salir por HTTPS al puerto 443 hacia Supabase. No abre ningún puerto de entrada, porque es él quien pregunta y no espera a que nadie se le conecte. Esto me pareció una ventaja importante en redes de hospital, donde normalmente no se permite abrir puertos.

El último punto son los antivirus. Un PowerShell oculto que corre con permisos altos puede parecerles sospechoso. La forma de evitar bloqueos es firmar los scripts y coordinarse con el área de sistemas del cliente, tal como lo prevé la tabla de riesgos del protocolo. También habrá que probarlo en equipos con reglas de control de aplicaciones, porque ahí PowerShell puede trabajar con un lenguaje limitado.

3. RESULTADOS DE LA ACTIVIDAD

El resultado de esta actividad son los requerimientos del agente en Windows. Los separé en dos grupos: los **funcionales**, que dicen qué tiene que hacer el agente, y los **no funcionales**, que dicen cómo tiene que hacerlo en cuanto a seguridad, rendimiento y compatibilidad.

Tabla 6. Requerimientos funcionales del agente en Windows.

Clave	Requerimiento	Descripción	Prioridad
RF-W01	Revisar el estado del equipo	Obtener uso de procesador, memoria, disco, procesos, servicios y tiempo encendido.	Alta
RF-W02	Enviar el reporte periódico	Mandar cada 5 minutos el reporte en el formato definido en la actividad A1.5.	Alta
RF-W03	Consultar comandos pendientes	Preguntar a la plataforma si hay una ejecución pendiente para su equipo.	Alta
RF-W04	Ejecutar comandos del catálogo	Correr en PowerShell únicamente el script del catálogo que recibe.	Alta
RF-W05	Simular comandos	Si la ejecución viene en modo de simulación, describir lo que haría sin ejecutarlo.	Alta
RF-W06	Enviar la salida en vivo	Mandar la salida por grupos de líneas mientras el comando se ejecuta.	Alta
RF-W07	Informar el resultado	Cerrar cada ejecución con su código de salida y su estado final.	Alta
RF-W08	Guardar reportes sin conexión	Conservar los reportes que no se pudieron enviar y mandarlos al volver la red.	Media
RF-W09	Registrar lo que hace	Llevar un registro local de las ejecuciones para auditoría.	Media

Fuente: Elaboración propia.

Tabla 7. Requerimientos no funcionales del agente en Windows.

Clave	Tipo	Requerimiento	Cómo se comprobará
RNF-W01	Compatibilidad	Funcionar con Windows PowerShell 5.1 en Windows 10 22H2, Windows 11 y Windows Server 2016 o posterior, sin instalar programas.	Matriz de compatibilidad.
RNF-W02	Disponibilidad	Ejecutarse como tarea programada aunque nadie haya iniciado sesión y reiniciarse solo si falla.	Reiniciar el equipo y revisar el historial de la tarea.
RNF-W03	Seguridad	No modificar la política de ejecución de todo el equipo.	Revisar la política antes y después de instalar.

Clave	Tipo	Requerimiento	Cómo se comprobará
RNF-W04	Seguridad	Proteger la clave del equipo con permisos solo para SYSTEM y administradores.	Revisar los permisos de la carpeta.
RNF-W05	Seguridad	Usar solo conexiones HTTPS de salida con TLS 1.2 o superior.	Revisión del código y de la red.
RNF-W06	Rendimiento	Usar menos del 5 % del procesador en promedio y menos de 2 minutos por ejecución.	Medición en el piloto (H5).
RNF-W07	Confiabilidad	Reintentar con espera creciente, hasta 60 segundos, cuando falle la red.	Desconectar la red durante una ejecución.
RNF-W08	Portabilidad	Tomar las métricas del sistema y no del texto traducido, para que funcione en cualquier idioma.	Prueba en un equipo en inglés.

Fuente: Elaboración propia.

También dejé lista la matriz de compatibilidad que se va a llenar durante las pruebas.

Tabla 8. Matriz de compatibilidad por completar en las pruebas.

Equipo	Versión	PowerShell	Resultado
Computadora Windows del practicante	Windows 10 pro	5.1	Usada en la demostración de la consola en vivo
Equipo de laboratorio de Bitlab-IT	Windows 11	5.1	Pendiente
Equipo de laboratorio de Bitlab-IT	Windows 10 22H2	5.1	Pendiente
Servidor de prueba	Windows Server 2019 o 2022	5.1	Pendiente

Fuente: Elaboración propia.

4. PENDIENTES Y SIGUIENTES PASOS

- Conocer las versiones de Windows que tienen los clientes, cuando tenga acceso a la plataforma de monitoreo.
- Validar con la asesora externa el uso de la cuenta SYSTEM y si se firmarán los scripts para producción.
- Confirmar si los clientes del piloto usan proxy o directivas de grupo que fijen la política de ejecución.
- Contar con equipos de laboratorio para llenar la matriz de compatibilidad.

Con esto cerrado, la siguiente actividad es la A1.2, donde hago el mismo análisis para las computadoras Mac.

REFERENCIAS

Bitlab-IT. (2026). Protocolo de estadía profesional (versión 1.0).

Microsoft. (2024a). PowerShell documentation: About execution policies. Microsoft Learn. https://learn.microsoft.com/powershell/

Microsoft. (2024b). ScheduledTasks module: Task Scheduler. Microsoft Learn. https://learn.microsoft.com/powershell/module/scheduledtasks/